HAL open science

A Step-by-Step Shor's Algorithm Walkthrough: from Theory to Implementation

Trung Nguyen, Samuel Petruescu, Shrikant Jadhav, Hiu Yung Wong

▶ **To cite this version:**

Trung Nguyen, Samuel Petruescu, Shrikant Jadhav, Hiu Yung Wong. A Step-by-Step Shor's Algorithm Walkthrough: from Theory to Implementation. 2026. ⟨hal-05541791v1⟩

HAL Id: hal-05541791

https://hal.science/hal-05541791v1

Preprint submitted on 7 Mar 2026 (v1), last revised 10 Mar 2026 (v2)

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.

Distributed under a Creative Commons CC BY-NC-ND 4.0 - Attribution - Non-commercial use - No Derivative Works - International License

A Step-by-Step Shor's Algorithm Walkthrough: from Theory to Implementation

Trung Nguyen, Samuel Petruescu, Shrikant Jadhav, and Hiu Yung Wong*

Department of Electrical Engineering,

San Jose State University, San Jose, CA 95192

(Dated: March 7, 2026)

Abstract

Quantum computing is inherently abstract and often counterintuitive from the perspective of classical physics. For learners to develop a deeper understanding, it is useful to study a practical example thoroughly, from its mathematical formalism to quantum circuit implementation. The Shor's algorithm for period finding, which can then be used for prime factorization, is demonstrated in this paper due to its practical significance.

While numerous publications on Shor's algorithm exist, they tend to focus on advanced theoretical analysis, specific implementation details, or narrowly defined research aspects; consequently, they are often not well-suited for educational purposes for beginning and intermediate learners. This paper presents a rigorous step-by-step walkthrough of Shor's algorithm, utilizing the conventional approach, and discusses the construction of the quantum oracle with illustrative numerical examples. This is followed by classical simulations and implementations on actual quantum hardware. The aim is to allow learners with minimal mathematical background to follow and understand the algorithm. Through this process, the paper also attempts to interpret and interconnect the key underlying quantum computing concepts of superposition, entanglement, interference, quantum oracle, and phase kick-back concepts that are often assumed to be trivial and omitted in research papers, yet are particularly challenging for novice learners to internalize.

In the numerical examples, examples with reduced qubit numbers are included. But by contrasting the results from the reduced qubit number setup with the standard one, learners can better appreciate the key enablers in Shor's algorithm and how the accuracy decreases with a smaller number of qubits. Moreover, reduced qubit examples (e.g., 5 qubits) make the example small enough to be implemented in a hardware emulator, such as a low-cost and low-capacity Field Programmable Gate Array (FPGA), which can be used in experimental hardware classes for students to understand how to use an FPGA to simulate quantum circuits.

I. INTRODUCTION

As demonstrated by Shor in his famous 1994 paper [1], a quantum computer can factor large integers in polynomial time, representing an exponential speedup over classical ap-

* hiuyung.wong@sjsu.edu

proaches. This breakthrough poses a significant challenge to current cryptographic schemes, such as 2048-bit RSA, which rely on the presumed intractability of large integer factoring.

Shor's approach to integer factorization can be briefly described in the following steps (Fig. 1): a) given an odd number N (the number to factorize), choose a random number a such that a and N are co-prime, i.e., their greatest common divisor, gcd, is 1 ($gcd\ (a, N) = 1$); b) find the order (or period) R, which is the smallest positive integer satisfying $a^R \pmod N \equiv 1 \pmod N$ (or $a^R - 1 \pmod N \equiv 0 \pmod N$). Here, modular arithmetic is used, and it means that both sides have the same remainders when they are divided by N. This is equivalent to finding the period of the function $f(x) = a^x \pmod N$. It should be noted that per number theory, for any co-prime (a, N), $f(x)$ is always periodic; c) if an even R is found, then $a^R - 1 = (a^{R/2} + 1) \cdot (a^{R/2} - 1)$; and if $a^R + 1$ is not divisible by N, the non-trivial factors of N are obtained by computing $gcd\ (a^{R/2} \pm 1, N)$. In case an odd R is found, the algorithm fails and needs to be restarted with a different a. So integer factorization is reduced essentially to the period-finding problem, which can be solved efficiently using quantum computation.

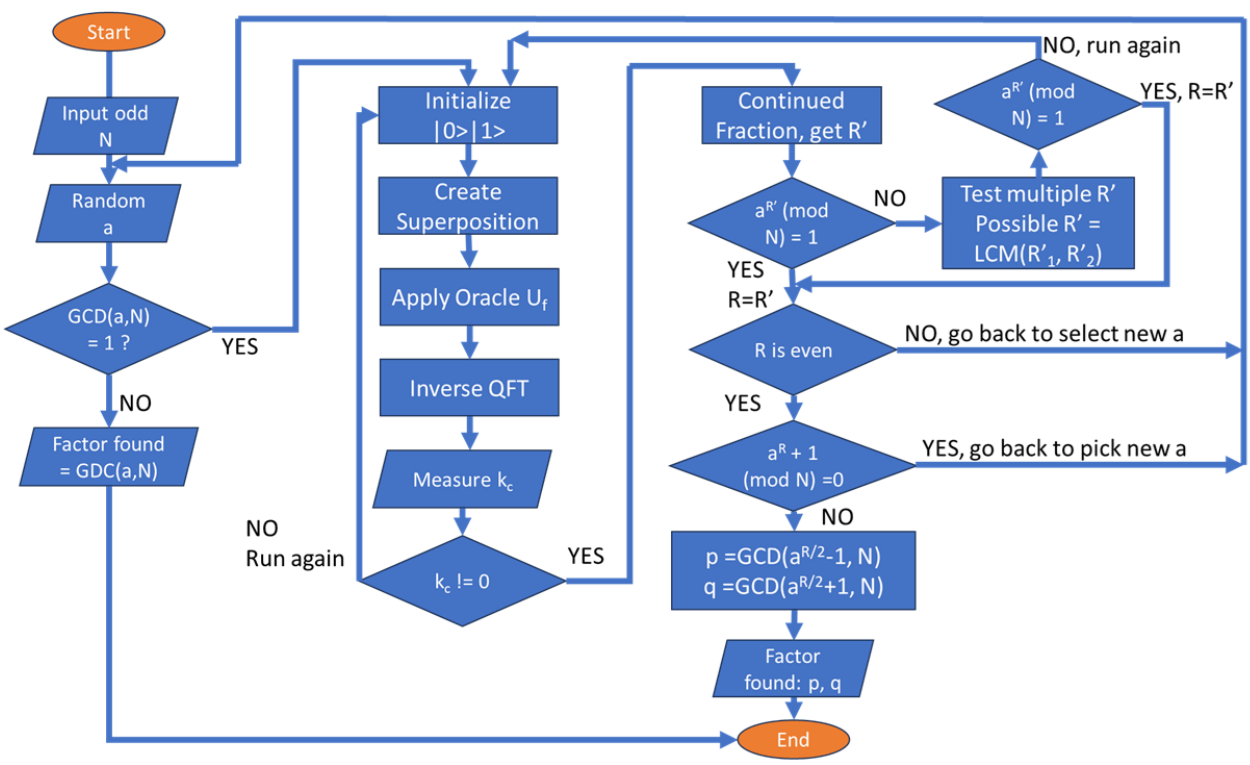


FIG. 1: Flowchart of Shor's Algorithm for Prime Factorization

Shor's algorithm for period-finding was originally discovered using modular exponentia-

tion and the quantum Fourier transform (QFT). However, it is discussed in most quantum computing textbooks [2–5] as well as in a recent pedagogical paper (2023) [6] as a specific case of quantum phase estimation (QPE)—an advanced quantum computing concept that is far more abstract and mathematically heavy than the algorithm itself. Although this approach provides a robust and concise mathematical framework for error analysis, it often discourages learners with a less extensive mathematical foundation or causes them to lose sight of the quantum principles behind the algorithm after navigating the complex math.

In this paper, we work through the quantum computation for solving period-finding by employing the conventional approach of modular exponentiation and the QFT. Our objective is to minimize abstract mathematical concepts and emphasize the quantum interpretation and implementation, making the material more accessible and beneficial to quantum computing learners.

The paper first presents the high-level quantum circuit for order-finding and traces the state-vector evolution at each step analytically. After that, a numerical example for $N = 15$ is presented to illustrate the state-vector evolution in the quantum circuit. Next, the quantum oracle and its implementations are discussed to demystify the "black box". A quantum phase estimation is then briefly related as a different way to interpret Shor's algorithm. The subsequent two sections detail Python simulations designed to reinforce and broaden theoretical understanding and demonstrate implementations on actual IBM quantum hardware for various N and a, helping learners close the loop between theory, simulation, and physical implementation.

A. How to use this paper

This paper is intended for learners who are already familiar with basic linear algebra and basic quantum concepts, such as qubit vector representations, superposition, entanglement, bra–ket notation, matrix multiplication, quantum gates, quantum oracles, and QFT. Some prior exposure to Python programming and IBM-Q Qiskit is necessary, but can be acquired in parallel. Readers may refer to the Appendix of [7] to refresh the fundamental concepts.

The paper may be used for self-study, as an assigned reading, or as a course module to guide the learners through the theoretical derivation of the algorithm, followed by hands-on Python simulations and IBM-Q implementations to analyze and verify the results. For

advanced learners or class projects, it can serve as a reference that can be replicated and extended by modifying parameters (different N and a) in both simulation and quantum circuit implementations.

II. SHOR'S ALGORITHM THEORETICAL WALKTHROUGH

A. Overview

Let us start with a quantum circuit block diagram for Shor's algorithm as shown in Fig. 2. It should be noted that, following IBM's convention, the most significant bit (MSB) is at the bottom, while the least significant bit (LSB) is at the top. It has $3n$ qubits, which are grouped into two registers. The first register ($2n$ qubits) initialized to $|0\rangle$ and the second register of n qubits is initialized to $|1\rangle$. $n = \lceil \log_2 N \rceil$ and N is the number to be factorized. Typically, n is chosen to satisfy $2^n >= N$. As mentioned in Section I, the goal is to find the period, R, of the function $f(x) = a^x \pmod N$.

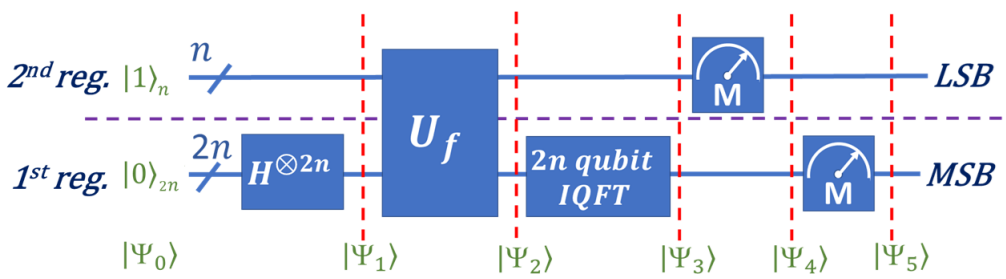


FIG. 2: Quantum circuit representation of Shor's Algorithm

A quantum state can be represented by its state vector. The algorithm can be viewed as an evolution of the state vector from Ψ_0 to Ψ_5 across each successive operation. Following the state initialization $|\Psi_0\rangle$, the Hadamard gates are applied to put the first register into a uniform superposition of all basis states, and the Quantum Oracle, which performs the modular exponentiation operation and is defined by,

$$U_f \, |x\rangle_{2n} \, |1\rangle_n = |x\rangle_{2n} \, |1 \cdot a^x \pmod N\rangle_n \tag{1}$$

is applied to the whole circuit. After that, the Inverse Quantum Fourier Transform (IQFT) is applied to the first register, followed by measurements of the second and first registers to retrieve information about the period, which is then used to compute the period and find the

factors of N. Note that, in principle, the measurement of the second register is not necessary for the algorithm to function properly. However, it is kept for pedagogical reasons.

To understand how the algorithm and its corresponding quantum circuit work, in Section II B, we will go through the mathematical derivation of the state vector evolution at each step to help us appreciate the properties of the quantum circuit. A numerical example will be shown in Section II C to help us understand the derivations and concepts in Section II B further.

B. Mathematical Derivation of State Vector Evolution

Step 0: State Vector Initialization

The state vector is initialized as:

$$|\Psi_0\rangle = |0\rangle^{\otimes 2n} |0\rangle^{\otimes n-1} |1\rangle = |0\rangle_{2n} |1\rangle_n . \tag{2}$$

This is a $3n$ qubit system and is partitioned into two registers: the first register at the most significant $2n$ bits (MSB) and the second register at the least significant n bits (LSB).

Step 1: Superposition Creation

Applying Hadamard gates to all qubits in the first register prepares a uniform superposition over all computational basis states of the first register from 0 to $2^{2n} - 1$.

$$\begin{aligned} |\Psi_1\rangle &= H^{\bigotimes 2n} \otimes I^{\otimes n} |\Psi_0\rangle = H^{\otimes 2n} |0\rangle_{2n} I^{\otimes n} |1\rangle_n \\ &= \frac{1}{\sqrt{2^{2n}}} \sum_{x=0}^{2^{2n}-1} |x\rangle_{2n} |1\rangle_n = \frac{1}{2^n} \sum_{x=0}^{2^{2n}-1} |x\rangle_{2n} |1\rangle_n . \end{aligned} \tag{3}$$

This superposition represents the coherent coexistence of all basis states of the first register with a single basis state ($|1\rangle$) of the second register. For example, when $n = 1$, the Hadamard gate is applied to the first register of $2n = 2$ qubits, the resulted statevector is:

$$|\Psi_1\rangle = \frac{1}{2^1} \sum_{x=0}^{2^2-1} |x\rangle_2 |1\rangle_1 = \frac{1}{2}(|00\rangle |1\rangle + |01\rangle |1\rangle + |10\rangle |1\rangle + |11\rangle |1\rangle). \tag{4}$$

This superposition consists of 4 basic states $\{|001\rangle , |011\rangle , |101\rangle , |111\rangle\}$ to collapsed to with an equal probability of 1/4 upon measurement.

Step 2: Application of the Quantum Oracle

Note that here we just apply the oracle U_f by its mathematical definition. Its construction will be discussed in Section III. Due to the linearity property, the oracle acting on Ψ_1 can be applied independently to each of its summation terms. Using Eq. (1) for the definition of U_f,

$$\begin{aligned} |\Psi_2\rangle = U_f\,|\Psi_1\rangle &= U_f(\frac{1}{2^n}\sum_{x=0}^{2^{2n}-1}|x\rangle_{2n}\,|1\rangle_n), \\ &= \frac{1}{2^n}\sum_{x=0}^{2^{2n}-1}|x\rangle_{2n}\,|1\cdot a^x \pmod N\rangle_n\,, \\ &= \frac{1}{2^n}\sum_{x=0}^{2^{2n}-1}|x\rangle_{2n}\,|a^x \pmod N\rangle_n\,, \\ &= \frac{1}{2^n}\sum_{x=0}^{2^{2n}-1}|x\rangle_{2n}\,|f(x)\rangle_n\,. \end{aligned} \tag{5}$$

Since the goal is to find the period, R, of the function $f(x) = a^x \pmod N$, now we will discuss how the period of $f(x)$ has affected the state vector $|\Psi_2\rangle$. Since $f(x) = a^x \pmod N$ has a period of R, which means $f(x) = f(x+R)$, for every possible x in the set $X = \{0, 1, ..., 2^{2n}-1\}$ in the first register, each remainder value $d_r = a^x \pmod N$ in the second register keeps repeating periodically in the order of $\{d_0 = 1, d_1, ..., d_{R-1}\}$ with period R (whereas $r \in [0, R-1]$) is the index of the remainders). Therefore, there are only R possible states in the second register, and each of them is a remainder, which are $|d_0 = 1\rangle$, $|d_1\rangle$, $\cdots$, $|d_{R-1}\rangle$. Each of them ($|d_r\rangle$) is associated with a superposition state (a part of the full superposition of the 2^n basis states in Eq. (5) in the first register), which is:

$$|X_r\rangle = |r\rangle + |r+1\cdot R\rangle + |r+2\cdot R\rangle + \cdots + |r+(L-1)\cdot R\rangle = \sum_{l=0}^{L-1}|r+l\cdot R\rangle\,, \tag{6}$$

whereas L is the number of times the remainder d_r repeats for x in $X = \{0, 1, ..., 2^{2n}-1\}$. Note that L depends on d_r and thus r, i.e., $L = L(r)$. For example, for $r = 0$, the second register has $|d_0 = 1\rangle$ and associated superposition state in the first register is $|0\rangle + |0+1\cdot R\rangle + |0+2\cdot R\rangle + \cdots + |0+(L-1)\cdot R\rangle$. And the combined state is $|X_0\rangle = (|0\rangle + |R\rangle + |2R\rangle \cdots + |(L-1)R\rangle) \otimes |1\rangle$.

Therefore, we can express the first register in $|\Psi_2\rangle$ as,

$$\sum_{x=0}^{2^{2n}-1} |x\rangle_{2n} = \sum_{r=0}^{R-1} |X_r\rangle_{2n} = \sum_{r=0}^{R-1} \left(\sum_{l=0}^{L(r)-1} |r + l \cdot R\rangle_{2n} \right) \tag{7}$$

And the total $|\Psi_2\rangle$ is,

$$|\Psi_2\rangle = \frac{1}{2^n} \sum_{r=0}^{R-1} |X_r\rangle_{2n} |d_r\rangle_n = \frac{1}{2^n} \sum_{r=0}^{R-1} \left(\sum_{l=0}^{L(r)-1} |r + l \cdot R\rangle_{2n} \right) |d_r\rangle_n \tag{8}$$

In other words, the oracle U_f creates entanglements among each basis state $|d_r\rangle$ in the second register and a superposition state $|X_r\rangle$ in the first register. Let us go a bit deeper in analyzing the value of $L(r)$. We can express $2^{2n} = q \cdot R + b$, with $b < R$. In the case $b = 0$, every remainder has the same L with $L = q$. But if $b \neq 0$ then value of L will depend on its remainders: $L(r) = q + 1$ for $r < b$, and $L(d_r) = q$ for $r \geq b$. So the index L in Eq. (6) to Eq. (8) may not be the same for all d_r and may be q or $q + 1$. However, for large n, this difference becomes very small and practically L is a constant with $L = \lfloor 2^{2n}/R \rfloor$.

We note that the period R is embedded into the superposition pattern of each $|X_r\rangle$ (i.e., in the term $|r + l \cdot R\rangle_{2n}$). However, it can't be retrieved by measuring the first register at this step because the first register is still in a uniform superposition of all basis states and thus will collapse to any value of $|x\rangle$ with equal probability upon measurement.

Step 3: Apply the Inverse Quantum Fourier Transform (IQFT)

IQFT for a m qubit statevector $|x\rangle_m$ is defined as:

$$IQFT \ |x\rangle_m = \frac{1}{\sqrt{2^m}} \sum_{k=0}^{2^m-1} \omega^{-x \cdot k} |k\rangle_m \quad \text{whereas} \quad \omega = e^{i \cdot 2\pi/2^m} \tag{9}$$

Note that some literature defines this as QFT [5]. By applying IQFT to the $2n$ qubits of the first register, the resulting state vector is:

$$
\begin{aligned}
|\Psi_3\rangle &= (IQFT_{2n} \otimes I^{\otimes n})\ |\Psi_2\rangle = (IQFT_{2n} \otimes I^{\otimes n}) \left(\frac{1}{2^n} \sum_{r=0}^{R-1} \left(\sum_{l=0}^{L-1} |r + l \cdot R\rangle_{2n} \right) |d_r\rangle_n \right) \\
&= \frac{1}{2^n} \sum_{r=0}^{R-1} \left(\sum_{l=0}^{L-1} (IQFT_{2n}\ \ |r + l \cdot R\rangle_{2n} \right) |d_r\rangle_n \\
&= \frac{1}{2^n} \sum_{r=0}^{R-1} \left(\sum_{l=0}^{L-1} \frac{1}{\sqrt{2^{2n}}} \sum_{k=0}^{2^{2n}-1} \omega^{-(r+l\cdot R)k} |k\rangle_{2n} \right) |d_r\rangle_n \,. \qquad (10)
\end{aligned}
$$

By swapping the orders of the two inner summations and rearranging the exponentiation terms of ω, the state vector $|\Psi_3\rangle$ can be expressed as:

$$
|\Psi_3\rangle = \frac{1}{2^{2n}} \sum_{r=0}^{R-1} \sum_{k=0}^{2^{2n}-1} \omega^{-r\cdot k} \left(\sum_{l=0}^{L-1} \left(\omega^{-R}\right)^{l\cdot k} \right) |k\rangle_{2n} |d_r\rangle_n \,. \qquad (11)
$$

Now let us try to observe the probability distribution of the first register of $|\Psi_3\rangle$ at different k. The most inner summation $S = \sum_{l=0}^{L-1}((\omega^{-R})^{l\cdot k}$, which is the sum of the $L-$th root of unity (by recognizing $\omega^{-R} = e^{-i\cdot 2\pi\cdot R/2^{2n}} \approx e^{-i\cdot 2\pi/L}$ for large values of n), will periodically experience constructive interferences at k equal to multiples of L and destructive interferences at other values, as will be discussed in Step 5. Since k runs from 0 to 2^{2n-1}, the constructive interferences will occur for $2^{2n}/L \approx R$ times. Therefore, for each $|d_r\rangle$ in the second register, it is now associated with R peaks in the control register, which are equally spread out at a distance L over the range of $k \in (0, 2^{2n} - 1)$.

Here we see that IQFT has transformed very entangled pair $|X_r\rangle_{2n} |d_r\rangle_n$ of the state vector $|\Psi_2\rangle$ in (Eq. 8), where $|X_r\rangle$ is in the uniform superposition state, into the new state where its corresponding first register state concentrates into a superposition of only R peaks. The R peaks in the first register for each $|d_r\rangle$ are at $|0\rangle$, $|L\rangle$, $|2L\rangle$, $\cdots$, $|(R-1)L\rangle$ with each peak carrying its own phase of $\omega^{-r\cdot k}$, for $k = 0, L, 2L, \cdots, (R-1)L$.

Step 4: Measuring the second register

Therefore, after IQFT, for each $|d_r\rangle$, its entangled first register state already contains the period information. So, we can perform a measurement on the second register to let it collapse to any state to further extract the period. When the second register is measured,

it collapses to a specific remainder $d_{r_c} \in \{d_0, d_1, \ldots, d_{R-1}\}$, and the state vector becomes:

$$|\Psi_4\rangle \approx \sum_{k=0}^{2^{2n}-1} \omega^{-r_c \cdot k} \cdot \sum_{l=0}^{L-1} \left(\omega^{-R}\right)^{l \cdot k} |k\rangle_{2n} |d_{r_c}\rangle_n , \tag{12}$$

where state normalization after the measurement is not performed for simplicity and, thus, $\approx$ is used.

Step 5: Measuring the first register

When the first register of $|\Psi_4\rangle$ is measured, its state collapses to a specific value k_c:

$$|\Psi_5\rangle \approx \omega^{-r_c \cdot k_c} \cdot \left(\sum_{l=0}^{L-1} \left(\omega^{-R}\right)^{l \cdot k_c} \right) |k_c\rangle_{2n} |d_{r_c}\rangle_n . \tag{13}$$

To understand the probability of measuring a specific k_c, we evaluate the amplitude of the summation terms in Eq. (13) by rearranging it:

$$S = \sum_{l=0}^{L-1} \left(\omega^{-R}\right)^{l \cdot k_c} = \sum_{l=0}^{L-1} \left(\omega'\right)^{l \cdot k_c} \quad \text{whereas} \quad \omega' = \omega^{-R} = e^{-i \cdot \frac{2\pi \cdot R}{2^{2n}}} \approx e^{-i \cdot \frac{2\pi}{L}}. \tag{14}$$

This summation of $L-$th roots of unity will peak when $k_c = z \cdot L$ whereas $z \in (0, 1, \ldots, R-1)$:

$$S = \sum_{l=0}^{L-1} (e^{-i \cdot \frac{2\pi}{L} \cdot z \cdot L})^l = \sum_{l=0}^{L-1} (e^{-i \cdot 2\pi \cdot z})^l = \sum_{l=0}^{L-1} (1)^l = L. \tag{15}$$

And for at $k_c \neq z \cdot L$, the summation of geometrical series becomes:

$$S = \frac{1 - (\omega'^{k_c})^L}{1 - \omega'^{k_c}} = \frac{1 - (\omega'^L)_c^k}{1 - \omega'^{k_c}} = \frac{1 - 1^{k_c}}{1 - \omega'^{k_c}} = 0. \tag{16}$$

Therefore, by measurement, the first register will collapse only to one of its probability peaks k_c where constructive interference occurs, or each measured $|k_c\rangle$ must satisfy:

$$k_c = z \cdot L = z \cdot 2^{2n}/R. \tag{17}$$

For $k_c \neq 0$, the period R can be determined by $R = z \cdot 2^{2n}/k_c$. If the measured $k_c = 0$, R cannot be determined, the measurement is considered failed, and the quantum algorithm has to be rerun. Please note that we have simplified the derivation with approximation of $L \cdot R = 2^{2n}$ or R divides 2^{2n} perfectly. These are perfect cases where the period R happens to be a power of 2, such as $N = 15, 51, \ldots$. In imperfect cases, R does not divide 2^{2n} and L is

not a single value, Eq. (14), Eq. (15) and Eq. (16) result in approximations, the constructive interferences, therefore, are not at single peaks but spread out and $z \cdot 2^{2n}/k_c$ is a factional number. In these cases, to calculate R, the continued fractions algorithm is employed as elaborated in [2].

C. Numerical Example for N=15

To reinforce the understanding of Shor's algorithm and the analytical derivation in the previous subsections, we will go through a numerical example with $N = 15$. This is the smallest practical number to demonstrate Shor's period finding algorithm. We chose $n = 4$ qubits for the second register (so that $N = 15 < 2^4$), $2n = 8$ qubits for the first register and a random number of $a = 7$, which is co-prime to N, i.e., $\gcd(7, 15) = 1$.

First, we calculate the modular sequence $f(x) = a^x \pmod N$:

$$
\begin{aligned}
&f(0) = 7^0 \pmod{15} = 1 \\
&f(1) = 7^1 \pmod{15} = 7 \\
&f(2) = 7^2 \pmod{15} = 49 \pmod{15} = 4 \\
&f(3) = 7^3 \pmod{15} = 343 \pmod{15} = 13 \\
&f(4) = 7^4 \pmod{15} = 1 \\
&\quad\vdots
\end{aligned}
$$

The sequence of remainders is $d_r = \{1, 7, 4, 13\}$ with period $R = 4$, which is assumed to be unknown, and is to be found.

Step 0: State Vector Initialization

The initial state vector, as shown in Eq. (2):

$$|\Psi_0\rangle = |00000000\rangle\,|0001\rangle\,. \tag{18}$$

Step 1: Superposition Creation

Applying Hadarmard gate to all qubits of the first register, as shown in Eq. (3):

$$|\Psi_1\rangle = \frac{1}{2^4}\sum_{x=0}^{2^8-1} |x\rangle_8 |1\rangle_4 = \frac{1}{16}(|0\rangle|1\rangle + |1\rangle|1\rangle + |2\rangle|1\rangle + ... + |255\rangle|1\rangle). \tag{19}$$

Note that the decimal representation of the states is used. The first register now is in uniform superposition states of all basis states $|0\rangle, |1\rangle, \ldots, |255\rangle$.

Step 2: Application of the Quantum Oracle

Apply the oracle U_f and regroup the terms, as in Eq. (5), Eq. (6), Eq. (7) and Eq. (8):

$$\begin{aligned}|\Psi_2\rangle &= \frac{1}{16}\sum_{x=0}^{255} |x\rangle_8 |f(x)\rangle_4 = \frac{1}{16}\big(|0\rangle|1\rangle + |1\rangle|7\rangle + |2\rangle|4\rangle + |3\rangle|13\rangle + \cdots + |255\rangle|13\rangle\big) \\ &= \frac{1}{16}\Bigg(\underbrace{\big(|0\rangle + |4\rangle + |8\rangle \ + ... + |252\rangle\big)}_{|X_0\rangle,\ L=64\ terms} |1\rangle + \underbrace{\big(|1\rangle + |5\rangle + |9\rangle \ + ... + |253\rangle\big)}_{|X1\rangle,\ L=64\ terms} |7\rangle + \\ &\qquad \underbrace{\big(|2\rangle + |6\rangle + |10\rangle \ + ...|254\rangle\big)}_{|X_2\rangle,\ L=64\ terms} |4\rangle + \underbrace{\big(|3\rangle + |7\rangle + |11\rangle \ + ... + |255\rangle\big)}_{|X_3\rangle,\ L=64\ terms} |13\rangle\Bigg).\end{aligned} \tag{20}$$

Here we can observe that after applying U_f, the second register is transformed from state $|1\rangle$ to a superposition state of all remainders: $|1\rangle, |7\rangle, |4\rangle, |13\rangle$ and each of these states is entangled with the corresponding superposition state $|X_0\rangle, |X_1\rangle, |X_2\rangle, |X_3\rangle$ in the first register.

Step 3: Application of the IQFT

Applying the IQFT, as per definition in Eq. (9) into each X_i, we have:

$$IQFT_8\,|X_0\rangle = IQFT_8\big(|0\rangle + |4\rangle + ... + |252\rangle\big) = \frac{1}{\sqrt{256}}\sum_{k=0}^{255}\big(\omega^{-k\cdot 0} + \omega^{-k\cdot 4} + ... + \omega^{-k\cdot 252}\big)\,|k\rangle \tag{21}$$

$$\begin{aligned} IQFT_8\,|X_1\rangle &= IQFT_8\big(|1\rangle + |5\rangle + ... + |253\rangle\big) = \frac{1}{\sqrt{256}}\sum_{k=0}^{255}\big(\omega^{-k\cdot 1} + \omega^{-k\cdot 5} + ... + \omega^{-k\cdot 253}\big)\,|k\rangle \\ &= \frac{1}{\sqrt{256}}\sum_{k=0}^{255}\omega^{-1\cdot k}\big(\omega^{-k\cdot 0} + \omega^{-k\cdot 4} + ... + \omega^{-k\cdot 252}\big)\,|k\rangle \end{aligned} \tag{22}$$

$$\begin{aligned} IQFT_8\,|X_2\rangle &= IQFT_8\big(|2\rangle + |6\rangle + ... + |254\rangle\big) = \frac{1}{\sqrt{256}}\sum_{k=0}^{255}\big(\omega^{-k\cdot 2} + \omega^{-k\cdot 6} + ... + \omega^{-k\cdot 254}\big)\,|k\rangle \\ &= \frac{1}{\sqrt{256}}\sum_{k=0}^{255}\omega^{-2\cdot k}\big(\omega^{-k\cdot 0} + \omega^{-k\cdot 4} + ... + \omega^{-k\cdot 252}\big)\,|k\rangle \end{aligned} \tag{23}$$

$$\begin{aligned} IQFT_8\,|X_3\rangle &= IQFT_8\big(|3\rangle + |7\rangle + ... + |255\rangle\big) = \frac{1}{\sqrt{256}}\sum_{k=0}^{255}\big(\omega^{-k\cdot 3} + \omega^{-k\cdot 7} + ... + \omega^{-k\cdot 255}\big)\,|k\rangle \\ &= \frac{1}{\sqrt{256}}\sum_{k=0}^{255}\omega^{-3\cdot k}\big(\omega^{-k\cdot 0} + \omega^{-k\cdot 4} + ... + \omega^{-k\cdot 252}\big)\,|k\rangle\,, \end{aligned} \tag{24}$$

whereas $\omega = e^{i\cdot 2\pi/256}$.

By evaluating the inner summation of ω in Eq. (21) to Eq. (24):

$$S = \omega^{-k\cdot 0} + \omega^{-k\cdot 4} + ... + \omega^{-k\cdot 252} = (\omega^{-4})^{k\cdot 0} + (\omega^{-4})^{k\cdot 1} + ... + (\omega^{-4})^{k\cdot 63}. \tag{25}$$

We can see that S is summation of 64th-roots of unity with $\omega' = \omega^{-4} = e^{i\cdot 2\pi/64}$, which the constructive interference at $k = 0,\ 64,\ 128,\ 192$ (mulitples of $L = 64$) resulting in $S = 64$. And it has destructive interference for all other k values, resulting in $S = 0$. Therefore, removing the terms with $S = 0$, and replacing $S = 64$ for the non-zero terms, the state vector, as in Eq. (11), can be simplified as:

$$\begin{aligned} |\Psi_3\rangle &= \frac{1}{16}\Big(IQFT_8\,|X_0\rangle\,|1\rangle + IQFT_8\,|X_1\rangle\,|7\rangle + IQFT_8\,|X_2\rangle\,|4\rangle + IQFT_8\,|X_3\rangle\,|13\rangle\Big) \\ &= \frac{1}{16}\cdot\frac{1}{16}\Big(\big(64\cdot|0\rangle + \omega^{-0\cdot 64}\cdot 64\cdot|64\rangle + \omega^{-0\cdot 128}\cdot 64\cdot|128\rangle + \omega^{-0\cdot 192}\cdot 64\cdot|192\rangle\big)\ |1\rangle \\ &+ \big(64\cdot|0\rangle + \omega^{-1\cdot 64}\cdot 64\cdot|64\rangle + \omega^{-1\cdot 128}\cdot 64\cdot|128\rangle + \omega^{-1\cdot 192}\cdot 64\cdot|192\rangle\big)\ |7\rangle \\ &+ \big(64\cdot|0\rangle + \omega^{-2\cdot 64}\cdot 64\cdot|64\rangle + \omega^{-2\cdot 128}\cdot 64\cdot|128\rangle + \omega^{-2\cdot 192}\cdot 64\cdot|192\rangle\big)\ |4\rangle \\ &+ \big(64\cdot|0\rangle + \omega^{-3\cdot 64}\cdot 64\cdot|64\rangle + \omega^{-3\cdot 128}\cdot 64\cdot|128\rangle + \omega^{-3\cdot 192}\cdot 64\cdot|192\rangle\big)\ |13\rangle\Big) \end{aligned} \tag{26}$$

Replacing $\omega^{-64} = e^{-i.2\pi\cdot 64/256} = e^{-i\pi/2} = -i$ we have:

$$|\Psi_3\rangle = \frac{1}{4}\Bigg(\big(|0\rangle + |64\rangle + |128\rangle + |192\rangle\big)\ |1\rangle + \big(|0\rangle - i\,|64\rangle - |128\rangle + i\,|192\rangle\big)\ |7\rangle \\ + \big(|0\rangle - |64\rangle + |128\rangle - |192\rangle\big)\ |4\rangle + \big(|0\rangle + i\,|64\rangle - |128\rangle - i\,|192\rangle\big)\ |13\rangle\Bigg). \tag{27}$$

It can be observed from $|\Psi_3\rangle$ in Eq. (26) and Eq. (27) that each joint state of the first and second registers has a different phase, which is now revealed by IQFT. These phases actually come from the second register (to be explained in the quantum phase estimation discussion in Section III C) and become relative phases of the entangled joint states, or are said to be "kicked back" to the first register.

Step 4: Measuring the second register

From Eq. (27), we can clearly see the effect of constructive and destructive interferences, which concentrate the states of the first register to the superpositions of 4 basis states: $|0\rangle$, $|64\rangle$, $|128\rangle$, $|192\rangle$. And each state $\{|1\rangle, |7\rangle, |4\rangle, |13\rangle\}$ of the second register is entangled with a superposition of $\{|0\rangle, |64\rangle, |128\rangle, |192\rangle\}$ basis states of the first register. Therefore, when performing a measurement, regardless of what value the second register collapses to when measured, the first register will collapse to a superposition of $\{|0\rangle, |64\rangle, |128\rangle, |192\rangle\}$. For example, if the second register collapses to $d_r = 7$, the state vector (normalized) collapses to:

$$|\Psi_4\rangle = \frac{1}{2}\big(|0\rangle - i\,|64\rangle - |128\rangle + i\,|192\rangle\big)\ |7\rangle\,. \tag{28}$$

Step 5: Measuring the first register

When the first register is measured, the state vector in Eq. (28) will collapse to one of $k_c \in 0, 64, 128, 192$ with an equal probability of 25%. Suppose it collapses to $k_c = 192$, the state vector becomes $|\Psi_5\rangle = |192\rangle\,|7\rangle$ and we have $R = z \cdot 256/192 = z \cdot 4/3$ and R can be determined using the continued fraction algorithm (as shown in Python code example in [17]).

An interesting observation is that the state vector $|\Psi_3\rangle$ in Eq. (27) can be rearranged to reveal the entanglement between the non-zero basis states of the first register and the

superpositions of the second register, as shown in Eq. (29). Clearly, if the first register is measured directly after step 3, it will collapse to one of the peaks k_c, anyway. Therefore, measuring the second register in Step 4 can be omitted.

$$\begin{aligned}|\Psi_3\rangle = \frac{1}{4}\Big(&|0\rangle\ \big(|1\rangle + |7\rangle + |4\rangle + |13\rangle\big)\ +\ |64\rangle\ \big(|1\rangle - i\,|7\rangle - |4\rangle + i\,|13\rangle\big)\\ &+\ |128\rangle\ \big(|1\rangle - |7\rangle + |4\rangle - |13\rangle\big)\ +\ |192\rangle\ \big(|1\rangle + i\,|7\rangle - |4\rangle - i\,|13\rangle\big)\Big)\end{aligned} \tag{29}$$

III. ORACLE IN SHOR'S ALGORITHM

A quantum oracle is often discussed at an abstract level as a "black box" that can perform a defined function, without specifying how it can be constructed for a given problem. This makes the oracle concept elusive for beginners, particularly when it is constructed by using the solution of the function to be solved. In this section, we will discuss how to construct the oracle in Shor's algorithm systematically based on its arithmetic properties and demonstrate a practical implementation that takes advantage of the known period of $f(x)$, followed by a discussion of Quantum Phase Estimation.

A. High level Oracle Construction

Let us examine the oracle in Eq. 1, rewritten here for convenience:

$$U_f\,|x\rangle_{2n}\,|1\rangle_n = |x\rangle_{2n}\,|1\cdot a^x \pmod N\rangle_n\,. \tag{30}$$

Its function can be described as taking the value $|x\rangle$ of the first register to multiply with the current value of the second register that was initialized to $|y\rangle = |1\rangle$ and then computing the modular exponentiation $a^x \pmod N$ and store the result back into the second register $|y\rangle$. The Toffoli gate, which is a universal gate for reversible classical logic, can be used as the fundamental building block to synthesize the oracle's arithmetic circuit.

A general construction of this oracle, following the architecture proposed by Beauregard [10], as illustrated in Fig 3, consists of a cascade of controlled-unitary gates, $C\text{-}U_{a^{2^i}}$, where as each gate applies a modular multiplication operation, $y\cdot a^{2^i} \pmod N$, to the second (target) register (i.e., $U_{a^{2^i}}\,|y\rangle = |y\cdot a^{2^i} \pmod N\rangle$), controlled by the ith qubit of the first (control) register.

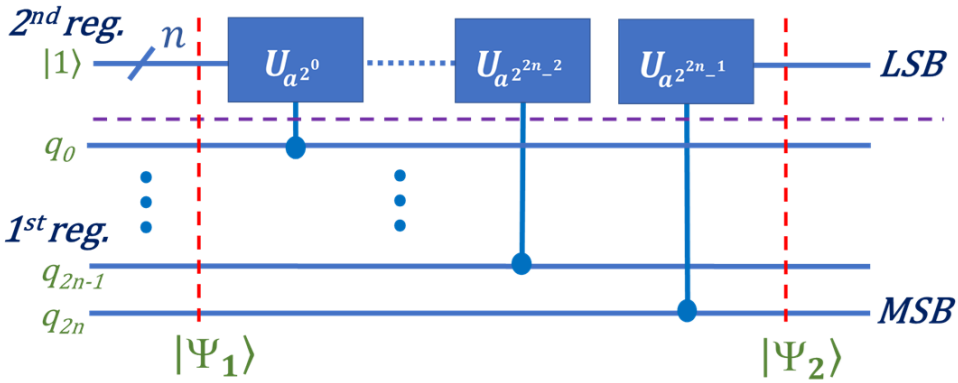


FIG. 3: High-level circuit for Shor's Oracle U_f

The circuit can be explained by decomposing the value of $|x\rangle$ into binary format $x = x_0.2^0 + x_1.2^1 + ... + x_{2n-1}.2^{2n-1}$. The operation of the oracle becomes,

$$\begin{aligned} U_f\,|x\rangle\,|1\rangle &= |x\rangle\,|1\cdot a^x \pmod N\rangle = |x\rangle\,|1\cdot a^{(x_0.2^0\ +\ x_1.2^1\ +\ldots+\ x_{2n}.2^{2n})} \pmod N\rangle \\ &= |x\rangle\,|\ldots\underbrace{\Big(\underbrace{\big(\underbrace{1\cdot(a^{2^0})^{x_0} \pmod N}_{C\text{-}U_{a^{2^0}}}\big)\cdot(a^{2^1})^{x_1} \pmod N}_{C\text{-}U_{a^{2^1}}}\Big)\cdots\cdot(a^{2^{2n-1}})^{x_{2n-1}}}_{C\text{-}U_{a^{2^{2n-1}}}} \pmod N\rangle \end{aligned} \tag{31}$$

We used the identity that $a\cdot b \pmod N = (a \pmod N)\cdot b \pmod N$. Each term $(a^{2^i})^{x_i}$ in Eq. (31) corresponds to the gate (C-$U_{a^{2^i}}$) in Fig. 3 with this control rule: when the i-th control qubit $x_i = 1$, the operator $U_{a^{2^i}}$ is applied to the target register and if $x_i = 0$, no change is made in the target register, as $(a^{2^i})^{x_i} = (a^{2^i})^0 = 1$. This results in a sequential chain of operations acting on the target register, with the intermediate value C-$U_{a^{2^i}}\,|y\rangle$ stored in the target register and used for the next multiplication step, as shown in Eq. (32).

$$|x\rangle|y=1\rangle \xrightarrow{C\text{-}U_{a^{2^0}}} |x\rangle|y = a^{x_0 2^0} \pmod N\rangle \xrightarrow{C\text{-}U_{a^{2^1}}} |x\rangle|y = a^{x_1 2^1}\cdot a^{x_0 2^0} \pmod N\rangle\ldots \tag{32}$$

The operation of the controlled unitary operator C-U_a for each controlled qubit $|q_i\rangle$ of the control register can be described by projecting $|q_i\rangle$ into the computational basis ($|0\rangle$ and $|1\rangle$) as,

$$C\text{-}U_{a^{2^i}}\,|q_i\rangle\,|y\rangle = \underbrace{|0\rangle\,\langle 0|\,|q_i\rangle \otimes I\,|y\rangle}_{\text{apply when}|q_i\rangle=|0\rangle} + \underbrace{|1\rangle\,\langle 1|\,|q_i\rangle \otimes U_{a^{2^i}}\,|y\rangle}_{\text{apply when }|q_i\rangle=|1\rangle},$$

where $C\text{-}U_{a^{2^i}} |q_i\rangle |y\rangle = |0\rangle \langle 0| |q_i\rangle \otimes I |y\rangle$ when $q_i = 0$ and $C\text{-}U_{a^{2^i}} |q_i\rangle |y\rangle = |1\rangle \langle 1| |q_i\rangle \otimes U_{a^{2^i}} |y\rangle$ when $q_i = 1$. Therefore, the operator $C\text{-}U_{a^{2^i}}$ can be expressed as,

$$C\text{-}U_{a^{2^i}} = |0\rangle \langle 0| \otimes I + |1\rangle \langle 1| \otimes U_{a^{2^i}} \tag{33}$$

Eq. (33) defines the controlled operation on the $n+1$ qubit subspace formed by the i-th control qubit and the n-qubit target register. It can be expressed in the full $3n$-qubit system space by explicitly accounting for the position of the i-th control qubit:

$$\widehat{C\text{-}U}_{a^{2^i}} = \left(I^{\otimes(2n-i)} \otimes |0\rangle \langle 0|_i \otimes I^{\otimes(i-1)} \otimes I^{\otimes n}\right) + \left(I^{\otimes(2n-i)} \otimes |1\rangle \langle 1|_i \otimes I^{\otimes(i-1)} \otimes U_{a^{2^i}}\right), \tag{34}$$

whereas, additional identity operators are applied to the $2n-i$ more significant qubits and the $i-1$ less significant qubits than the i-th control qubit.

Then the full oracle U_f can be expressed as:

$$U_f = \prod_{i=0}^{2n-1} \widehat{C\text{-}U}_{a^{2^i}} = \widehat{C\text{-}U}_{a^{2^{2n-1}}} \cdots \widehat{C\text{-}U}_{a^{2^1}} \widehat{C\text{-}U}_{a^{2^0}} \tag{35}$$

With the above analysis, the oracle can be systematically synthesized as a cascade of controlled modular multiplication operators $C\text{-}U_{a^{2^i}}$. These $C\text{-}U_a$ operators can be constructed using a network of elementary single-qubit and two-qubit gates, alongside intermediate ancillary qubits, to incrementally synthesize the arithmetic components such as adders, multipliers, as discussed in [8–10]. Such circuit realizations are notably complex, requiring a very large number of gates and qubit counts. For illustration, a 1-bit full adder alone can be achieved using four qubits and four reversible gates—specifically two Toffoli and two CNOT gates [11]. Given the limitations of current NISQ-era quantum systems, true implementations of Shor's algorithm using controlled operators with arithmetic operations remain impractical, even for small instances such as $N = 15$, as recently demonstrated by recent work in 2025[12] on an IBM quantum processor.

For conceptual clarity, it should be noted that in his original paper [1], Shor presented the algorithm using a general oracle form $U_f |x\rangle |0\rangle = |x\rangle |a^x \pmod N\rangle$ and implicitly demonstrated the implementation using XOR oracle $U_f |x\rangle |0\rangle = |x\rangle |0 \oplus a^x \pmod N\rangle$. However, implementing this form of oracle requires additional resources and gate operations to manage and uncompute the intermediate garbage information to reset the target register to the initial state $|0\rangle$ at every intermediate step. Consequently, most practical implementations, as discussed in [8–10] and modern literature, utilize the multiplicative oracle:

$U_f\,|x\rangle\,|1\rangle = |x\rangle\,|1 \cdot a^x \pmod N\rangle$, which directly evolves the state $|1\rangle$ through successive modular multiplications.

B. Oracle Implementation

Instead of constructing the controlled operator C-U through explicit arithmetic circuits, a compiled approach based on permutation mapping can be employed to reduce gate depth and qubit counts. In this approach, we utilize the prior known knowledge of the period R and the modular sequence $U_{a^{2^i}}$ to realize the behavior of the oracle. It is therefore not a legitimate implementation of Shor's algorithm, but widely used in current practice to facilitate simulation and experimental testing on quantum processors.

It can be observed that applying the matrix operator $U_{a^{2^i}}$ actually rotates the current state of the target register from $|y\rangle$ to $|y'\rangle$ whereas $y' = y \cdot a^{2^i} \pmod N$. Therefore, the behavior of the operator can be described by a permutation matrix that maps all possible values of y to y'. In other words, the permutation matrix maps state $|y\rangle$ to $|y'\rangle$. It should follow the following rules:

- For $0 \leq y < N$: assign $y' = (a^{2^i} \cdot y) \pmod N$, and we set the element of the permutation matrix at column y and row y' to 1. That is, $U_{a^{2^i}}[y', y] = 1$. This maps state $|y\rangle$ to $|y'\rangle$.

- For $N \leq y < 2^n$: assign $y' = y$, and set $U_{a^{2^i}}[y', y] = 1$. That means no rotation is performed.

In fact, $N \leq y < 2^n$ are considered "invalid" states, as the remainders must be strictly less than N (recalling that the second register is for storing remainders). However, to preserve the unitarity (reversibility) of the operator, these states are mapped to themselves.

To illustrate this concept, let us consider an example of $N = 15$ and $a = 7$, the resulting mapping is shown in Table I. The third column shows how the numbers between 0 and 15 are mapped to each other. When $i \geq 2$, the remainder is always 1 and all $|y\rangle$ map to themselves. Therefore, no control gate is required for those qubits.

In general, a permutation matrix can be constructed through established reversible-logic synthesis methods that directly yield quantum circuits composed of NOT, CNOT, and Toffoli gates, as demonstrated in [13]. More recently, explicit constructions for quantum circuits implementing arbitrary permutation matrices using multi-controlled Toffoli gates were presented in [14].

TABLE I: Permutation Cycles for Compiled Modular Exponentiation ($N = 15, a = 7$)

Bit (i)	$\mathbf{7^i}$ (mod **15**)	**Permutation Cycles** ($\|\mathbf{y}\rangle \rightarrow \|\mathbf{y'}\rangle = \|\mathbf{y} \cdot \mathbf{7^i}$ (mod **15**)$\rangle$)
$i = 0$	$7^{2^0} = 7$	$(1{\rightarrow}7{\rightarrow}4{\rightarrow}13{\rightarrow}1), (2{\rightarrow}14{\rightarrow}8{\rightarrow}11{\rightarrow}2), (3{\rightarrow}6{\rightarrow}9{\rightarrow}12{\rightarrow}3),$
		$(0{\rightarrow}0), (5{\rightarrow}5), (10{\rightarrow}10), (15{\rightarrow}15)$
$i = 1$	$7^{2^1} = 4$	$(1{\leftrightarrow}4), (2{\leftrightarrow}8), (3{\leftrightarrow}12), (6{\leftrightarrow}9), (7{\leftrightarrow}13), (11{\leftrightarrow}14),$
		$(0{\rightarrow}0), (5{\rightarrow}5), (10{\rightarrow}10), (15{\rightarrow}15)$
$i \geq 2$	$7^{2^i} = 1$	$(y \leftrightarrow y)$ all y map to themselves

However, for a particular permutation matrix, the corresponding quantum circuit may be efficiently realized by taking advantage of its specific permutation behaviors. In the case of $N = 15, a = 7$, as demonstrated by Markov and Saeedi [15], the above permutation matrices can be realized using a sequence of controlled-NOT gates and controlled-SWAP (Fredkin) gates to minimize circuit depth, as illustrated in Fig. 4. Readers are encouraged to check that this circuit maps the states according to Table I. For example, if $|y\rangle = |7\rangle = |0111\rangle$, the action due to q_0 ($x[0]$ in the figure) will cause it to change to $|0100\rangle = |4\rangle$. This is because after the CNOT gates, it becomes $|1000\rangle$. After the three SWAP gates, it becomes $|0001\rangle$, $|0010\rangle$, and $|0100\rangle = |4\rangle$, respectively.

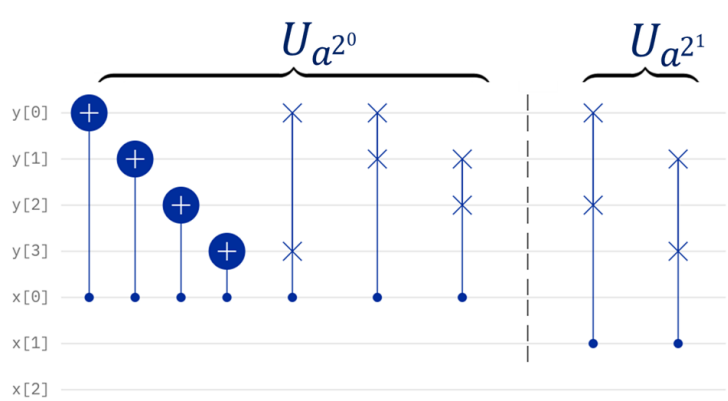


FIG. 4: The permutation oracle circuit for N=15, a = 7.

C. Shor's Algorithm as a Quantum Phase Estimation

By replacing the oracle in Fig. 2 by Fig. 3, the full quantum circuit of Shor's algorithm, as shown in Fig. 5, can be seen as the circuit of Quantum Phase Estimation (QPE) algorithm with the unitary operator $U = U_{a^{2^0}}$ and $U^{2^i} = U_{a^{2^i}}$, where the first register serves as the control and the second register as the target. Consequently, Shor's algorithm should be naturally interpreted from the perspective of quantum phase estimation. Please note that while Fig. 5 depicts the C-U operators being applied sequentially, they may be executed in any arbitrary order because these controlled operators commute [5, 7].

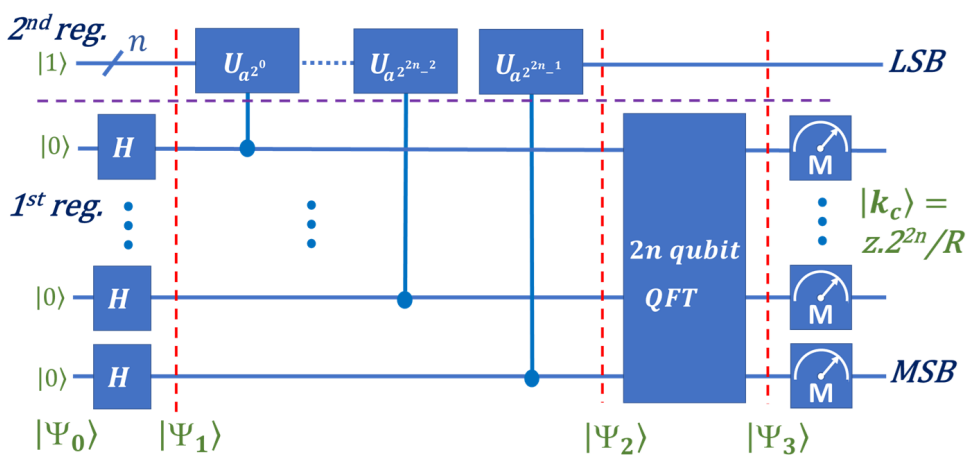


FIG. 5: Shor's Algorithm as Quantum Phase Estimation

QPE algorithm can be briefly described as: given a unitary operator U and an eigenvector $|u_z\rangle$ with eigenvalue $e^{2\pi i\theta_z}$ (i.e., $U|u_z\rangle = e^{2\pi i\theta_z}|u_z\rangle$), the phase $\theta_z \in [0, 1)$ can be estimated by storing the eigenvector $|u_z\rangle$ into the target register, applying the QPE circuit, and measuring the control register to estimate the phase.

For the Shor's algorithm in Fig. 5, the operator U and the resulting phase θ_z of QPE are defined by:

$$U|y\rangle = U_{a^{2^0}}|y\rangle = |y \cdot a \pmod N\rangle \tag{36}$$

$$\theta_z = \frac{z}{R} \approx \frac{k_c}{2^{2n}}, \tag{37}$$

where the vector under estimation is $|y\rangle = |1\rangle$, R is the number of eigenvectors in the same R-dimensional subspace of the unitary operator U, and $z \in \{0, 1, \ldots, R-1\}$ is the index of the eigenvectors. Here, we proceed by accepting the fact that the number of eigenvectors in

the subspace is identical to the period R of the modular exponentiation $a^x \pmod N$, and that the uniform superposition of these eigenvectors yields the state $|1\rangle$, as analyzed in detail in [2]. Therefore, applying QPE to $|y\rangle = |1\rangle$ yields a superposition of all phase estimates $\theta_z = z/R$ of all these eigenvectors. Subsequently, by measurement, the first register will collapse to a value k_c corresponding to one of the phases θ_z of the eigenvalues.

We illustrate the above analysis with the example $N = 15, a = 7$. The subspace of the operator U that maps $|y\rangle$ to $|y \cdot 7 \pmod{15}\rangle$ with initial $|y\rangle = |1\rangle$ is $\{1, 7, 4, 13\}$, as seen in Table I. Its eigenvectors with eigenvalue and phase are shown in Table II without derivation, but learners are encouraged to verify $\frac{1}{2}\big(|u_0\rangle + |u_1\rangle + |u_2\rangle + |u_3\rangle\big) = |1\rangle$ and $U\,|u_z\rangle = \lambda_z\,|u_z\rangle$ using Eq.(36).

TABLE II: Eigenvectors of the unitary operator U for $N = 15, a = 7$

Eigenvector	Computational Basis	Eigenvalue (λ_z)	Phase ($\theta_z = z/R$)
$|u_0\rangle$	$\frac{1}{2}(|1\rangle + |7\rangle + |4\rangle + |13\rangle)$	1	0
$|u_1\rangle$	$\frac{1}{2}(|1\rangle - i\,|7\rangle - |4\rangle + i\,|13\rangle)$	i	0.25 (1/4)
$|u_2\rangle$	$\frac{1}{2}(|1\rangle - |7\rangle + |4\rangle - |13\rangle)$	-1	0.50 (2/4)
$|u_3\rangle$	$\frac{1}{2}(|1\rangle + i\,|7\rangle - |4\rangle - i\,|13\rangle)$	$-i$	0.75 (3/4)

By decomposing $|y\rangle = |1\rangle$ as a superposition state of the eigenvectors $|u_z\rangle$, we can analyze the effect when applying controlled unitary operator $C\text{-}U_a$ in Eq. (33) to a control qubit $|q_i\rangle$ in a superposition state (i.e., after the Hadamard gate) and a target register in the eigenvector state $|u_i\rangle$ as follows:

The combined initial state is:

$$|\psi_{\mathrm{i}}\rangle = |q_i\rangle \otimes |u_z\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \otimes |u_z\rangle = \frac{1}{\sqrt{2}}(|0\rangle|u_z\rangle + |1\rangle|u_z\rangle) \tag{38}$$

After the application of the controlled operator, i.e., substituting $U|u_z\rangle = \lambda_z|u_z\rangle$, with the eigenvalue $\lambda_z = e^{2\pi i\theta_z}$, and $U_{a^{2^i}} = (U)^{2^i}$ into Eq. (38), the combined state becomes:

$$\begin{aligned} |\psi_i{}'\rangle &= C\text{-}U_{a^{2^i}}|\psi_i\rangle = \frac{1}{\sqrt{2}}\big(|0\rangle|u_z\rangle + |1\rangle U_{a^{2^i}}|u_z\rangle\big) \\ &= \frac{1}{\sqrt{2}}\big(|0\rangle|u_z\rangle + (\lambda_z)^{2^i}|1\rangle|u_z\rangle\big) = \frac{1}{\sqrt{2}}\big(|0\rangle + e^{2\pi i\theta_z\cdot 2^i}|1\rangle\big) \otimes |u_z\rangle \end{aligned} \tag{39}$$

From Eq. (39), we observe that the target register remains unchanged in state $|u_z\rangle$, while the i-th control qubit $|q_i\rangle$ acquires a relative phase shift of $e^{2\pi i\theta_z 2^i}$. Consequently, since the initial state $|y\rangle = |1\rangle$ is a superposition of all eigenvectors $|u_z\rangle$, each component induces a distinct phase shift to every qubit in the control register. This phenomenon, whereby the eigenvalues of the unitary operator U are mapped onto the control register, is the mechanism of phase kickback.

From Eq. (37), the measurement has a phase error of $|\frac{z}{R} - \frac{k_c}{2^{2n}}|$. As analyzed in [2], in order for the period R to be accurately recovered via the continued fraction algorithm, this phase error must satisfy:

$$\left|\frac{z}{R} - \frac{k}{2^{2n}}\right| \leq \frac{1}{2^{2n+1}} < \frac{1}{2R^2}. \tag{40}$$

From Eq. (17) and Eq. (37), it follows that for the specific cases where the period is a power of two ($R = 2^p$), the phase is measured exactly with zero error. Consequently, the first register requires a minimum number of p qubits in order to represent the phase exactly, allowing the algorithm to yield the correct period R. In the general case, the first register is typically configured with $2n$ qubits to satisfy the error threshold in Eq. (40).

A subtle question arises: since U possesses a dimension of 2^n, it must mathematically possess 2^n eigenvectors. Why do only R eigenvectors appear in the results of the second register? The reason lies in the fact that the initial state $|1\rangle$ belongs to a specific R-dimensional subspace of U, as seen in Table I of the numeric example with $N = 15$. Other initial states, such as $|y\rangle = |2\rangle, |3\rangle \ldots$, belong to different subspaces that have zero overlap with the subspace of $|1\rangle$. Since the second register is initialized strictly to $|1\rangle$, only the eigenvectors spanning this specific R-dimensional sub-permutation participate in the phase estimation process. This insight shows that Shor's algorithm may, in principle, be realized using any of the disjoint cycles of U by initializing the second register $|y\rangle$ to any state within a chosen cycle. In practice, however, the structure of these cycles and their associated remainders are unknown, except for the value "1", which makes it the default initialization.

IV. PYTHON SIMULATION OF SHOR'S ALGORITHM

While there are many simulation examples for Shor's algorithm that are well written and readily available such as on IBM Qiskit, Cirq, Github, it is worth for learners to go through the process of basic algorithm implementation by implementing their own simulator (which

is just a matrix multiplier) and run simulations for different values of N and a to gain more thorough understanding.

A. Simulation Implementation in Python

For this purpose, we first adopt a state-vector–based simulation that tracks the evolution of the quantum state directly, strictly following the mathematical derivations and analytical steps presented in Section II and Section III. For a system comprising $3n$ qubits, the corresponding state vector has a dimension of 2^{3n}. All gate operations are constructed as matrix operators of dimension $2^{3n} \times 2^{3n}$ using NumPy Kronecker (kron) matrix multiplication. During measurement, the state vector is collapsed according to the corresponding probabilities; therefore, NumPy's random choice function is used to simulate the measurement outcomes.

This approach requires constructing very large matrices, consuming substantial memory. On a standard laptop, the simulator can efficiently handle state vectors and matrices for up to 12 qubits (the standard requirement for $N = 15$). However, simulating $N = 21$ requires $3n = 15$ qubits, which may lead to crashes due to memory exhaustion. The code can be optimized, with insights from AI-assisted tools, in two key areas: (1) replacing large matrix-vector multiplications with in-place state-vector transformations, and (2) implementing the Inverse Quantum Fourier Transform (IQFT) using standard Inverse Fast Fourier Transform (IFFT) functions. These optimizations enable the simulation of larger cases, such as $N = 35$ or 57, with ease.

The simulator allows different values of (N, a) and n_c, n_t (number of qubits in the first and second registers) to be specified directly in the code to simulate different scenarios. Throughout the execution, the program displays the state vector evolution and plots the probability distribution for both registers. After measuring the control register value k_c, the simulator attempts to extract the period and calculate the prime factors. By adjusting n_c and n_t, users can also observe the effect of reducing the number of qubits on the performance.

Please note that the simulator follow the compiled approach, i.e., for a given (N, a), its unitary operators $U_{a^{2^i}}$ are constructed via the permutation mappings, as detailed in III A, instead of the full arithmetic circuit to realize the functions of the operator which are much more complex and resource-intensive.

The Python source codes of the following examples can be found at [16] and [17].

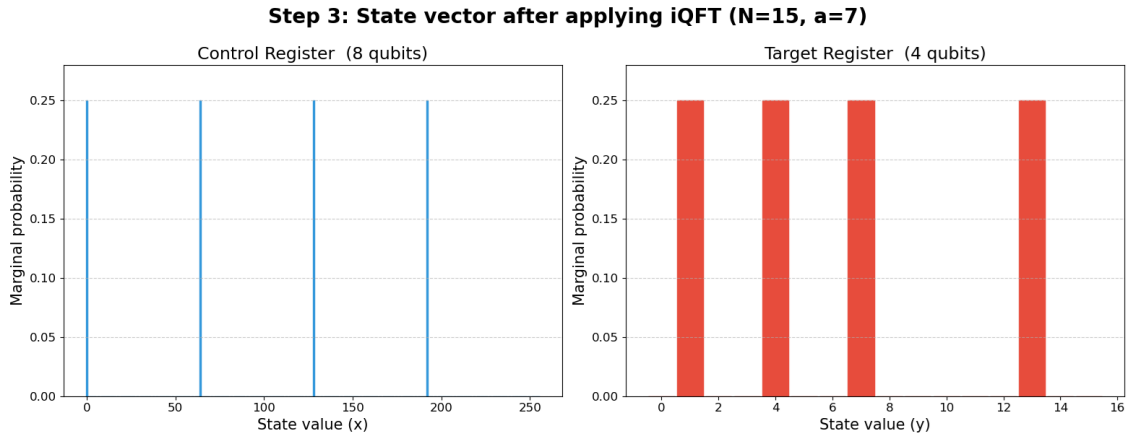
B. Simulation results for $N = 15$ and $N = 21$

As this simulation is noise-free, measurement outcomes align perfectly with the theoretical marginal probability distributions of the registers. Therefore, we directly plot these marginal probabilities for analysis rather than simulating measurements and plotting the measured outcomes.

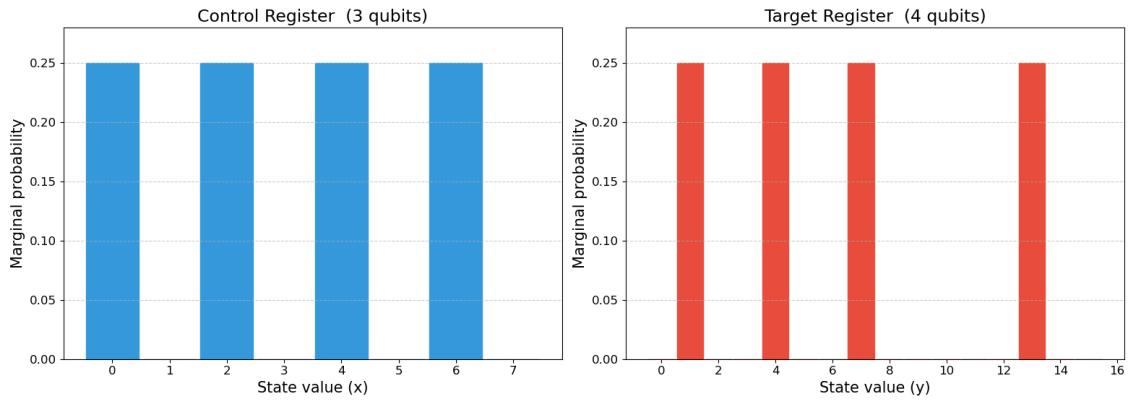
Firstly, we run simulations for $(N = 15, a = 7)$ with period $R = 4$, comparing two configurations: a standard implementation $(n_c = 8, n_t = 4)$, mirroring our numerical analysis in II C, and a reduced control qubit case of $n_c = 3$ qubits. Fig. 6 shows the results for the two implementation scenarios. The results of the probability peaks are observed exactly the same for both scenarios. Since the period $R = 4$ is a power of 2, the probability peaks are identical in both cases. Although for this unique value of R, n_c can theoretically be reduced to $\lceil \log_2(R) \rceil = 2$ for phase estimation with zero leakage, a larger size of $n_c = 3$ is chosen for pedagogical clarity to better visualize the periodic nature of the peaks. The specific behavior of the 7-bit register configuration is further demonstrated in Appendix A 1.

Next, simulations for $(N = 21, a = 11)$ with a period of $R = 6$ are performed to observe the probability distribution of k_c in the presence of phase estimation errors. Fig. 7 illustrates the results for two cases: (a) a standard configuration with $n_c = 10$, and (b) a reduced configuration with $n_c = 7$. In the reduced case, $n_c = 7$ is selected as the smallest value satisfying the error threshold in Eq. (40) required to decode the period R correctly. As observed, because the period $R = 6$ does not divide the register sizes (2^{10} or 2^7), the constructive interference peaks do not align with single integer values. Instead, the probability is spread across multiple neighboring states as spectral leakage. As seen in the plots, reducing the number of control qubits decreases the spectral resolution, causing this spreading to become more pronounced as the peaks widen.

Please note that in both examples above $(N = 15, 21)$, the measured k_c does not always yield the correct periods $(R = 4, 6)$, but may instead result in factors of R (e.g., $R = 2$ or 3). This occurs when the measured state k_c corresponds to a phase z/R where z and R share a common factor. This behavior is inherent to the algorithm; in such cases, the continued fraction algorithm extracts the reduced denominator rather than the true period. In practice, the full period is recovered by performing multiple trials and calculating the least common multiple (LCM) of the resulting candidates [1].



(a) The 1st register: 8 qubits, the 2nd register: 4 qubits

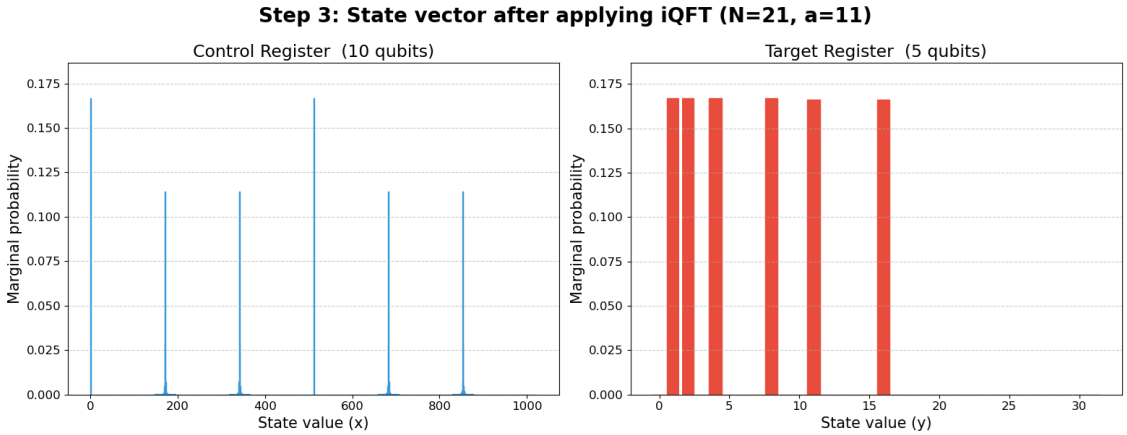


(b) The 1st register: 3 qubits, the 2nd register: 4 qubits

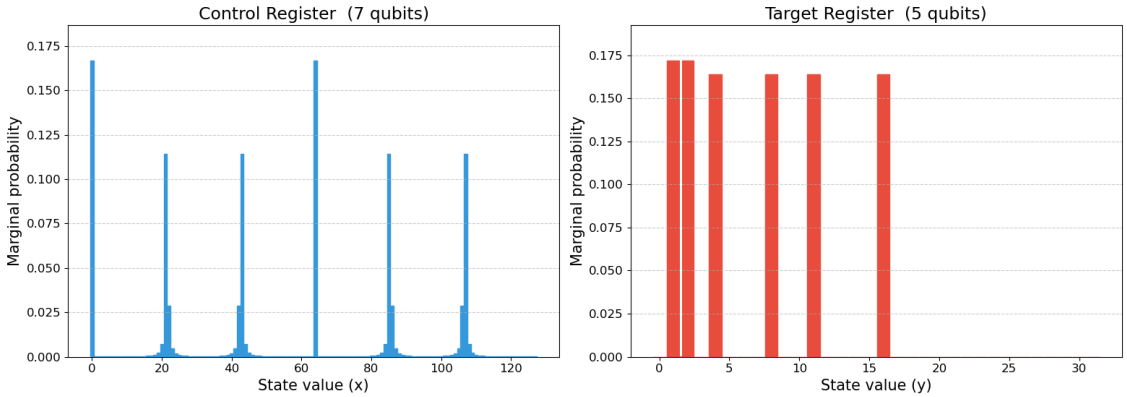
FIG. 6: Simulation results of marginal probability distribution for $N = 15, a = 7$ with different qubit sizes of the control register

Learners are encouraged to work out the corresponding permutation cycles for $N = 21, 35$ (as shown in Table I) and conduct simulations with various values for n_c and n_t. This allows for a deeper observation of how probability spreads and the computed period R behave under different register constraints.

Minimizing the qubit requirements for Shor's algorithm simulations remains highly desirable, even for small (N, a) cases. Reducing this resource overhead is essential for practical applications, such as emulation on low-cost FPGA boards or performance validation on physical quantum hardware, serving both research and pedagogical objectives.



(a) The 1st register: 10 qubits, the 2nd register: 5 qubits



(b) The 1st register: 7 qubits, the 2nd register: 5 qubits

FIG. 7: Simulation results of marginal probability distribution for $N = 21, a = 11$ with different qubit sizes of the control register

V. IMPLEMENTATION ON IBM QUANTUM HARDWARE

In this section, we explore the implementation of Shor's algorithm on physical quantum hardware. The IBM Quantum cloud-based platform was selected for its open-access tier, providing a practical environment for learners to implement and test the algorithm. Unlike the Python simulator described in Section IV, where operations are executed on classical computing resources, this implementation utilizes physical qubits and hardware-level quantum gates to synthesize the quantum circuit.

The primary objective is to demonstrate the algorithm's execution and verify experimental measurements against simulation results. We first examine the case $(N = 15, a = 7)$

using the reduced 7-qubit configuration discussed in Section IV B. Subsequently, we explore a 5-qubit implementation for $(N = 15, a = 4)$. By minimizing both gate depth and qubit counts relative to the 7-qubit case, this configuration enables a comparative performance evaluation of current Noisy Intermediate-Scale Quantum (NISQ) hardware. The Qiskit scripts used to construct these circuits and submit the jobs to IBM Quantum hardware are available in [16] and [17]. In order to run the codes, learners must create an account on IBM Cloud, generate a service token, and replace the existing token in the codes.

A. Quantum Circuit for $N = 15, a = 7$ using 7 Qubits

The quantum circuit is constructed as illustrated in Fig. 8, utilizing the permutation oracle shown in Fig. 4 and a standard 3-qubit IQFT circuit. The implementation was developed in Qiskit and executed on the IBM Quantum *ibm_fez* hardware using 1024 shots. The measured outcomes for the first register (x-register) are shown in Fig. 9. While the primary distribution peaks at $\{0, 2, 4, 6\}$ align with the simulation results in Fig. 6, there are non-zero counts for states that should theoretically be null. These extraneous counts indicate an experimental error rate of approximately 15%.

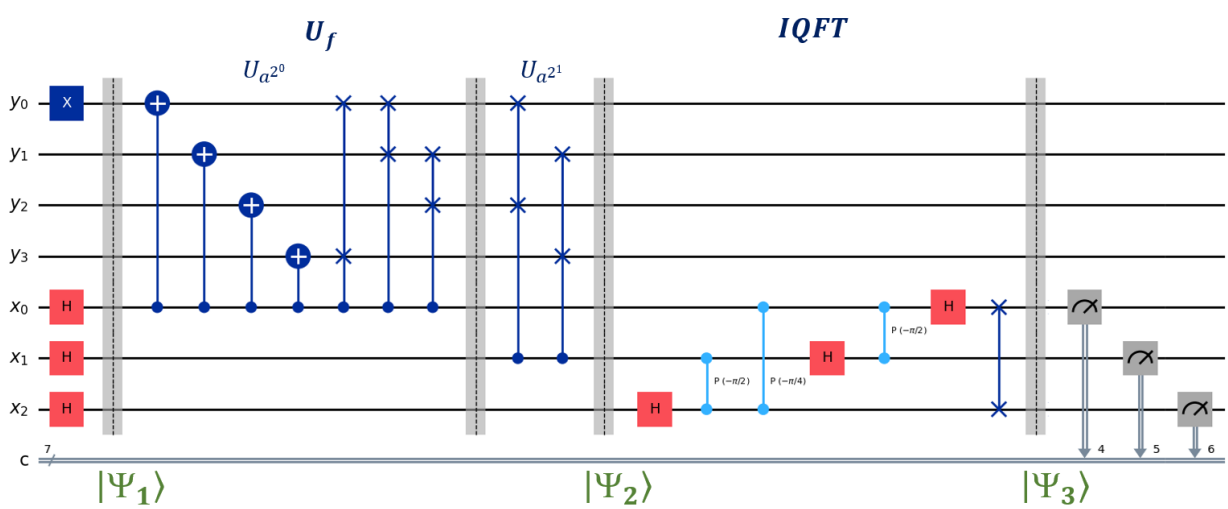


FIG. 8: 7-qubit Quantum circuit for Shor's algorithm (N=15, a = 7)

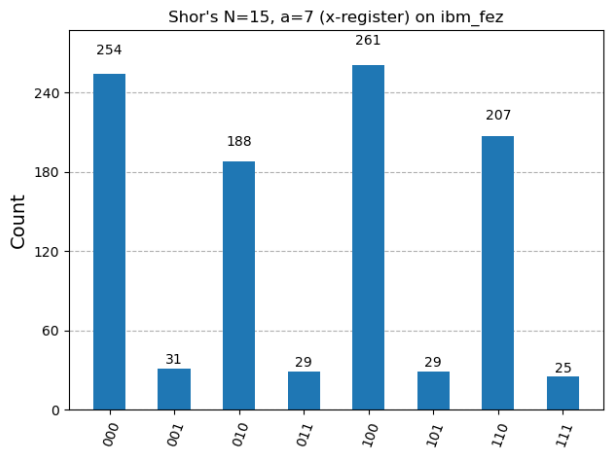


FIG. 9: Measurement results for N=15, a = 7 (7-qubit circuit)

B. Quantum Circuit for $N = 15, a = 4$ using 5 Qubits

The 5-qubit implementation is realized by utilizing 2-qubits for the first register ($n_c = 2$) to provide sufficient inputs to resolve the period $R = 2$ and 3-qubits for the second register to hold the modular remainder values $\{1, 4\}$. The mathematical derivation for this reduced configuration is provided in Appendix A 2. Learners are encouraged to reconstruct a permutation table, similar to Table I, to identify the distinct subspaces and conduct simulations with these parameters to verify the theoretical results.

Let us discuss the derivation of the permutation oracle U_f for this configuration. Because the remainders $\{1, 4\}$ can be represented using three qubits, we construct a modified permutation operator U_a that acts exclusively on the $\{1, 4\}$ subspace, mapping all other basis states to themselves. The resulting permutation mapping is:

$$(1 \leftrightarrow 4), (2 \rightarrow 2), (3 \rightarrow 3), (5 \rightarrow 5), (6 \rightarrow 6), (7 \rightarrow 7).$$

By observing the binary representation of the target register, $|001\rangle \leftrightarrow |100\rangle$, the operator U_a can be realized as a single controlled-SWAP (C-SWAP) gate between the first and third qubits of the second register.

The quantum circuit is shown in Fig. 10, consisting of the permutation oracle and a standard 2-qubit IQFT. The circuit is implemented in Qiskit and executed on IBM-Q *ibm_fez* hardware with 1024 shots. Measurement results are shown in Fig. 11, with clear peaks at 0 and 2 as expected, alongside small non-zero counts at 1 and 3. The error rate, estimated at approximately 2%, is significantly lower than the 15% observed in the 7-qubit implemen-

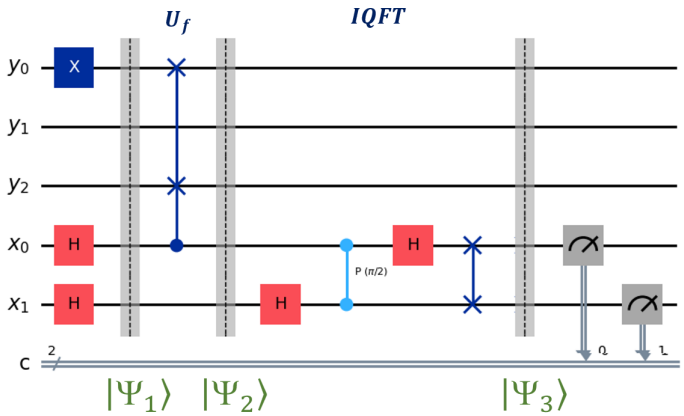


FIG. 10: A 5-qubit Quantum circuit for Shor's algorithm (N=15, a = 4)

tation. This improvement demonstrates that cumulative errors from decoherence and gate infidelity mount rapidly as gate depth and qubit counts increase [18].

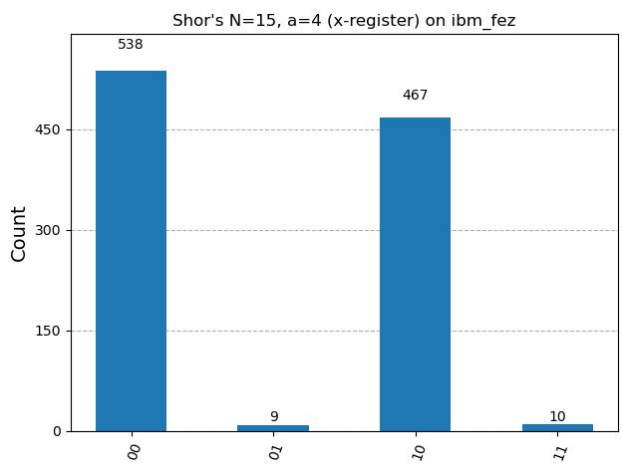


FIG. 11: Measurement results for N=15, a = 4 (5-qubit circuit)

It is important to note that the quantum circuit defined in Fig. 10 is a logical-level representation. To execute the algorithm on physical hardware, the Qiskit compiler must transpile these logical gates into a physical circuit compatible with the specific basis gate set and qubit connectivity of the target device. The resulting physical circuit, illustrated in Fig. 12, exhibits a more complex structure with a significantly higher count of physical gates. Interestingly, while the logical design specifies five qubits, the physical implementation utilizes only four; the qubit $|y_1\rangle$ is automatically optimized out by the compiler, as it remains in a constant state and does not participate in any operations.

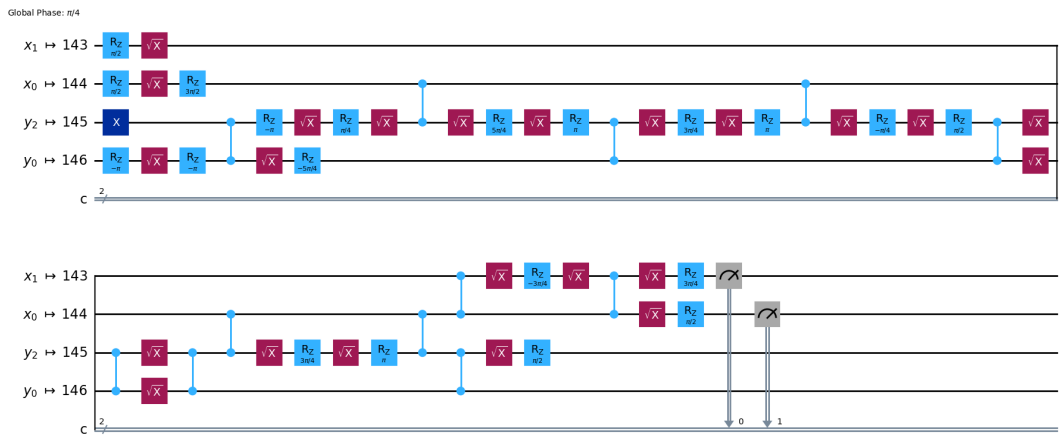


FIG. 12: Physical gate implementation of the 5-qubit quantum circuit

VI. DISCUSSION AND CONCLUSION

We have traced Shor's algorithm from its mathematical roots to actual execution on IBM Quantum hardware, but yet to interpret what makes this algorithm achieve such an exponential speedup over classical methods, and where does the quantum advantage actually come from?

From a circuit implementation perspective, many components of Shor's algorithm, such as modular arithmetic or IQFT, can be constructed with equivalent classical computing circuits and thus are not unique to the quantum approach. The quantum advantage emerges from superposition, which has no classical equivalent. This superposition property allows the quantum circuit to represent all possible input states simultaneously, enabling the computation to be performed "all at once." In contrast, classical computing can only process inputs sequentially, one at a time. In addition, for this process to succeed, entanglement plays a key role by correlating the first register with the results of modular arithmetic. This ensures that the periodic patterns of the second register are encoded into the phases of the first register, allowing the subsequent constructive and destructive interference to reveal the period R.

To gain deeper insight, one must recognize the key distinction between quantum superposition, which involves the superposition of probability amplitudes, and the superposition

of waves in classical physics, which involves the superposition of physical signals or displacements. A quantum computer can be seen as a probability machine that can produce and manipulate the probability states of quantum bits as well as create correlations among these probability states via entanglement.

In short, quantum superposition, entanglement, and interference are the enablers of exponential speedup of Shor's algorithm over classical computation and are the key elements in all quantum algorithms.

Appendix A: Derivation of State Vector for N=15 with Reduced Number of Qubits

1. Derivation of 7-Qubit State Vector Evolution for $N = 15, a = 7, n_c = 3, n_t = 4$

In this derivation, the first register uses 3 qubits to generate only two periods of remainders, and the second register uses 4 qubits to hold the remainders {1, 7, 4, 13}.

Step 0: State Vector Initialization

The initial state vector for $m = 3$ (instead of $2n = 8$) qubits in the first register and $n = 4$ in the second register is initialized as:

$$|\Psi_0\rangle = |000\rangle_3 |0001\rangle_4 = |0\rangle_3 |1\rangle_4 . \tag{A1}$$

Step 1: Superposition Creation

Applying the Hadamard gate to all qubits of the 3-qubit control register:

$$|\Psi_1\rangle = \frac{1}{\sqrt{2^3}} \sum_{x=0}^{2^3-1} |x\rangle_3 |1\rangle_4 = \frac{1}{\sqrt{8}} \sum_{x=0}^{7} |x\rangle |1\rangle = \frac{1}{\sqrt{8}}(|0\rangle |1\rangle + |1\rangle |1\rangle + \cdots + |7\rangle |1\rangle). \tag{A2}$$

The first register is now in a uniform superposition of all basis states from $|0\rangle$ to $|7\rangle$.

Step 2: Application of the Quantum Oracle

We apply the modular exponentiation oracle U_f for $f(x) = 7^x \pmod{15}$. The remainders for $x \in \{0, \ldots, 7\}$ repeat every $R = 4$ states ($\{1, 7, 4, 13\}$):

$$\begin{aligned}
|\Psi_2\rangle &= \frac{1}{\sqrt{8}} \sum_{x=0}^{7} |x\rangle_3 \, |f(x)\rangle_4 \\
&= \frac{1}{\sqrt{8}} \big(|0\rangle|1\rangle + |1\rangle|7\rangle + |2\rangle|4\rangle + |3\rangle|13\rangle + |4\rangle|1\rangle + |5\rangle|7\rangle + |6\rangle|4\rangle + |7\rangle|13\rangle \big) \\
&= \frac{1}{\sqrt{8}} \Bigg(\underbrace{\big(|0\rangle + |4\rangle\big)}_{|X_0\rangle} |1\rangle + \underbrace{\big(|1\rangle + |5\rangle\big)}_{|X_1\rangle} |7\rangle + \underbrace{\big(|2\rangle + |6\rangle\big)}_{|X_2\rangle} |4\rangle + \underbrace{\big(|3\rangle + |7\rangle\big)}_{|X_3\rangle} |13\rangle \Bigg).
\end{aligned} \tag{A3}$$

Each state $|X_j\rangle$ in the first register is now a superposition of $L = 2^3/4 = 2$ terms, entangled with the unique remainders in the second register.

Step 3: Application of the IQFT

Applying the $IQFT_3$ to each $|X_j\rangle$ with the 8th root of unity $\omega = e^{i2\pi/8}$:

$$\begin{aligned}
IQFT_3 \, |X_0\rangle &= \frac{1}{\sqrt{8}} \sum_{k=0}^{7} \big(\omega^{-k\cdot 0} + \omega^{-k\cdot 4}\big) \, |k\rangle = \frac{1}{\sqrt{8}} \sum_{k=0}^{7} (1 + \omega^{-4k}) \, |k\rangle \\
IQFT_3 \, |X_1\rangle &= \frac{1}{\sqrt{8}} \sum_{k=0}^{7} (\omega^{-k\cdot 1} + \omega^{-k\cdot 5}) \, |k\rangle = \frac{1}{\sqrt{8}} \sum_{k=0}^{7} \omega^{-k\cdot 1} (1 + \omega^{-4k}) \, |k\rangle \\
IQFT_3 \, |X_2\rangle &= \frac{1}{\sqrt{8}} \sum_{k=0}^{7} (\omega^{-k\cdot 2} + \omega^{-k\cdot 6}) \, |k\rangle = \frac{1}{\sqrt{8}} \sum_{k=0}^{7} \omega^{-k\cdot 2} (1 + \omega^{-4k}) \, |k\rangle \\
IQFT_3 \, |X_3\rangle &= \frac{1}{\sqrt{8}} \sum_{k=0}^{7} (\omega^{-k\cdot 3} + \omega^{-k\cdot 7}) \, |k\rangle = \frac{1}{\sqrt{8}} \sum_{k=0}^{7} \omega^{-k\cdot 3} (1 + \omega^{-4k}) \, |k\rangle
\end{aligned} \tag{A4}$$

Evaluating the summation term $S = 1 + \omega^{-4k}$: Since $\omega^{-4} = e^{-i(2\pi/8)\cdot 4} = e^{-i\pi} = -1$, the term becomes $S = 1 + (-1)^k$. This is the summation of the second-root ($2^m/R = 8/4 = 2$) of unity. Constructive interference occurs when k is even ($k \in \{0, 2, 4, 6\}$), where $S = 2$. Destructive interference occurs when k is odd, where $S = 0$.

Substituting these into the total state vector:

$$
\begin{aligned}
|\Psi_3\rangle = \frac{1}{\sqrt{8}} \cdot \frac{1}{\sqrt{8}} \Big(& (2\,|0\rangle + 2\,|2\rangle + 2\,|4\rangle + 2\,|6\rangle)\,|1\rangle \\
& + (2\omega^0\,|0\rangle + 2\omega^{-2}\,|2\rangle + 2\omega^{-4}\,|4\rangle + 2\omega^{-6}\,|6\rangle)\,|7\rangle \\
& + (2\omega^0\,|0\rangle + 2\omega^{-4}\,|2\rangle + 2\omega^{-8}\,|4\rangle + 2\omega^{-12}\,|6\rangle)\,|4\rangle \\
& + (2\omega^0\,|0\rangle + 2\omega^{-6}\,|2\rangle + 2\omega^{-12}\,|4\rangle + 2\omega^{-18}\,|6\rangle)\,|13\rangle \Big)
\end{aligned}
\tag{A5}
$$

Simplified with $\omega^{-2} = -i$, $\omega^{-4} = -1$, and $\omega^{-6} = i$:

$$
\begin{aligned}
|\Psi_3\rangle = \frac{1}{4} \Bigg(& \big(|0\rangle + |2\rangle + |4\rangle + |6\rangle\big)\,|1\rangle + \big(|0\rangle - i\,|2\rangle - |4\rangle + i\,|6\rangle\big)\,|7\rangle \\
& + \big(|0\rangle - |2\rangle + |4\rangle - |6\rangle\big)\,|4\rangle + \big(|0\rangle + i\,|2\rangle - |4\rangle - i\,|6\rangle\big)\,|13\rangle \Bigg).
\end{aligned}
\tag{A6}
$$

Measuring the second register, it will collapse to either $\Psi_{4_Reg2} = |1\rangle$, $|7\rangle$, $|4\rangle$, or $|13\rangle$, and Ψ_{4_Reg1} will remain in equal superposition of $0,\ 2,\ 4,\ 6$ with different phases. Measuring the first register will collapse to either $\Psi_{5_Reg1} = |0\rangle$, $|2\rangle$, $|4\rangle$, or $|6\rangle$, or measured $k_c = 0,\ 2,\ 4,\ 6$. By applying the formula $R = z.2^2/k_c$, the period still can be found $R = 4$ for $k_c = 2,\ 4,\ 6$.

2. Derivation of 5-Qubit State Vector Evolution for $N = 15, a = 4, n_c = 2, n_t = 3$

Step 0: State Vector Initialization

The initial state vector for $m = 2$ and $n = 3$ is initialized as:

$$
|\Psi_0\rangle = |00\rangle_2\,|001\rangle_3 = |0\rangle_2\,|1\rangle_3\,. \tag{A7}
$$

Step 1: Superposition Creation

Applying the Hadamard gate to the 2-qubit control register:

$$
|\Psi_1\rangle = \frac{1}{\sqrt{2^2}} \sum_{x=0}^{2^2-1} |x\rangle_2\,|1\rangle_3 = \frac{1}{2}(|0\rangle\,|1\rangle + |1\rangle\,|1\rangle + |2\rangle\,|1\rangle + |3\rangle\,|1\rangle). \tag{A8}
$$

Step 2: Application of the Quantum Oracle

We apply the oracle U_f for $f(x) = 4^x \pmod{15}$. The remainders for $x \in \{0, 1, 2, 3\}$ are $\{1, 4, 1, 4\}$:

$$\begin{aligned} |\Psi_2\rangle &= \frac{1}{2}\big(|0\rangle|1\rangle + |1\rangle|4\rangle + |2\rangle|1\rangle + |3\rangle|4\rangle\big) \\ &= \frac{1}{2}\Bigg(\underbrace{\big(|0\rangle + |2\rangle\big)}_{|X_0\rangle}|1\rangle + \underbrace{\big(|1\rangle + |3\rangle\big)}_{|X_1\rangle}|4\rangle\Bigg). \end{aligned} \tag{A9}$$

Here, each state $|X_j\rangle$ contains $L = 2^2/2 = 2$ terms.

Step 3: Application of the IQFT

Applying $IQFT_2$ to each $|X_j\rangle$ with the 4th root of unity $\omega = e^{i2\pi/4} = i$:

$$\begin{aligned} IQFT_2\,|X_0\rangle &= \frac{1}{\sqrt{4}}\sum_{k=0}^{3}\big(\omega^{-k\cdot 0} + \omega^{-k\cdot 2}\big)\,|k\rangle = \frac{1}{2}\sum_{k=0}^{3}(1 + \omega^{-2k})\,|k\rangle \\ IQFT_2\,|X_1\rangle &= \frac{1}{2}\sum_{k=0}^{3}\omega^{-k\cdot 1}(1 + \omega^{-2k})\,|k\rangle\,. \end{aligned} \tag{A10}$$

Evaluating the summation term $S = 1 + \omega^{-2k}$: Since $\omega^{-2} = e^{-i(2\pi/4)\cdot 2} = e^{-i\pi} = -1$, the term is $S = 1 + (-1)^k$. This is the summation of the second-root ($2^m/R = 4/2 = 2$) of unity. Constructive interference occurs at $k \in \{0, 2\}$ where $S = 2$. Destructive interference occurs at $k \in \{1, 3\}$ where $S = 0$.

Substituting these results into the total state vector:

$$\begin{aligned} |\Psi_3\rangle &= \frac{1}{2}\cdot\frac{1}{2}\Bigg(\big(2\,|0\rangle + 2\,|2\rangle\big)\,|1\rangle + \big(2\omega^0\,|0\rangle + 2\omega^{-2}\,|2\rangle\big)\,|4\rangle\Bigg) \\ &= \frac{1}{2}\Bigg(\big(|0\rangle + |2\rangle\big)\,|1\rangle + \big(|0\rangle - |2\rangle\big)\,|4\rangle\Bigg). \end{aligned} \tag{A11}$$

After the IQFT, due to constructive and destructive interference, the control register contains superpositions only of $|0\rangle$ and $|2\rangle$. Measuring the target register collapses it to either $|1\rangle$ or $|4\rangle$. Correspondingly, the post-measurement state of the control register is $|\Psi_{4,\text{Reg1}}\rangle = |0\rangle \pm |2\rangle$. A subsequent measurement of the control register collapses the state

to either $|\Psi_{5,\text{Reg1}}\rangle = |0\rangle$ or $|2\rangle$ with equal probability.

---

[1] P. W. Shor, "Algorithms for quantum computation: discrete logarithms and factoring," Proceedings 35th Annual Symposium on Foundations of Computer Science, 1994, pp. 124–134, doi: 10.1109/SFCS.1994.365700.

[2] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, Cambridge University Press, 2nd ed., 2010.

[3] E. Rieffel and W. Polak, *Quantum Computing: A Gentle Introduction*, MIT Press, Cambridge, MA, USA, 2011.

[4] P. Kaye, R. Laflamme, and M. Mosca, *An Introduction to Quantum Computing*, Oxford University Press, Oxford, UK, 2007.

[5] Hiu Yung Wong, Introduction to Quantum Computing: From a Layperson to a Programmer in 30 Steps. Switzerland: Springer Nature, 2022, pp. 170. https://doi.org/10.1007/978-3-030-98339-0. ISBN-10: 3030983382.

[6] R. L. Singleton Jr., *Shor's Factoring Algorithm and Modular Exponentiation Operators: A Pedagogical Presentation with Examples*, Preprint, ResearchGate, June 2023. Available at: `https://www.researchgate.net/publication/371606327`.

[7] A. Zaman, H. J. Morrell and H. Y. Wong, "A Step-by-Step HHL Algorithm Walkthrough to Enhance Understanding of Critical Quantum Computing Concepts," in IEEE Access, vol. 11, pp. 77117-77131, 2023, doi: 10.1109/ACCESS.2023.3297658.

[8] Vlatko Vedral, Adriano Barenco, and Artur Ekert, "Quantum networks for elementary arithmetic operations," Physical Review A, vol. 54, no. 1, pp. 147–153, 1996.

[9] David Beckman, Amalavoyal K. Chari, Srikant Devabhaktuni, and John Preskill, "Efficient networks for quantum factoring," Physical Review A, vol. 54, no. 2, pp. 1034–1063, 1996.

[10] Stéphane Beauregard, "Circuit for Shor's algorithm using $2n + 3$ qubits," Quantum Information & Computation, vol. 3, no. 2, pp. 175–185, 2003 (arXiv:quant-ph/0205095).

[11] A. Peres, "Reversible logic and quantum computers," *Physical Review A*, vol. 32, no. 6, pp. 3266–3276, 1985.

[12] B. Assel, "Where Are We with Shor's Algorithm? A deep dive into the implementation of Shor's algorithm and an analysis of quantum runs on IBM quantum hardware," *Towards Data*

Science, Jan. 2025. [Online]. Available: https://towardsdatascience.com/where-are-we-with-shors-algorithm/

[13] V. V. Shende, A. K. Prasad, I. L. Markov, and J. P. Hayes, "Synthesis of reversible logic circuits," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 22, no. 6, pp. 710–722, 2003.

[14] J. Hanson, "Quantum circuits for permutation matrices," arXiv:2512.11938, 2025.

[15] I. L. Markov and M. Saeedi, *Constant-Optimized Quantum Circuits for Modular Multiplication and Exponentiation*, Quantum Information & Computation, Vol. 12, No. 5&6, pp. 361–394, 2012.

[16] T. Nguyen, *Shor's-Algorithm-Simulation-in-Python-and-Qiskit*, (2026), GitHub repository, `https://github.com/ctn008/Shor-s-Algorithm-Simulation-in-Python-and-Qiskit`.

[17] *Shor's-Algorithm-Simulation-in-Python-and-Qiskit* GitHub repository, `https://github.com/hywong2/Shor-s-Algorithm-Simulation-in-Python-and-Qiskit`.

[18] A. Zaman and H. Y. Wong, "Study of Error Propagation and Generation in Harrow-Hassidim-Lloyd (HHL) Quantum Algorithm," 2022 IEEE Latin American Electron Devices Conference (LAEDC), Cancun, Mexico, 2022, pp. 1-4, doi: 10.1109/LAEDC54796.2022.9908231.